



PERFORMANCE EVALUATION OF DEEP LEARNING ARCHITECTURES FOR DETECTING CITRUS CROP HLB ABNORMALITIES

Nagineni Venkata Sireesha¹, Gillala Rekha^{*2}

¹Research Scholar, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation (Deemed to be University), Hyderabad, 500075, Telangana, India.

²Associate Professor, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, (Deemed to be University), Hyderabad, 500075, Telangana, India.

¹<https://orcid.org/0000-0003-3047-6387>^{id}, ²<https://orcid.org/0009-0003-4933-4255>^{id}

Email: sireeshagireesh@gmail.com, [*gillala.rekha@klh.edu.in](mailto:gillala.rekha@klh.edu.in)

ARTICLE INFO

Article History

Received: September 11, 2025

Reviewed: May 4, 2026

Accepted: June 16, 2026

Published: July 31, 2026

Keywords:

Citrus Huanglongbing (HLB),
Automated Disease Detection,
DCNN,
AlexNet architecture,
VGG19,
ResNet18,
Inception-V3

ABSTRACT

One of the most effective ways to combat diseases like Huanglongbing (HLB; aka citrus greening) is through early detection. Since HLB is a major disease causing significant losses in citrus production worldwide, there is a need for cheap and efficient methods for detecting it so that treatment can begin at the earliest stage. Deep learning techniques using convolutional neural networks (CNNs) are becoming the primary tools for disease detection in computer vision; however, it remains difficult to distinguish HLB from other citrus diseases. Diagnostic methods based on traditional techniques, e.g., quantitative real-time polymerase chain reaction, are costly and require large datasets to train CNN models. One way to tackle the lack of labelled images is to use transfer learning with pre-trained CNNs, which can also perform well on smaller datasets. The objective of the research is to evaluate (AlexNet, VGG19) the chain and the directed acyclic graph (DAG) CNN architectures (ResNet50, Inception-V3) for the three-class classification of Huanglongbing (HLB), healthy leaves, and 10 other citrus abnormalities. To do this, a dataset containing 955 colour images of Citrus leaves from southern India was used. Additionally, 10-fold cross-validation was performed. The experimental results showed that all the CNN models achieved a sensitivity of over 98% in detecting HLB. Yet VGG19 achieved the highest sensitivity score across all tests, despite having fewer parameters than Inception-V3. These findings indicate that models with more parameters are better able to handle the challenges posed by small datasets and, as a result, enable successful transfer learning for HLB detection. The present paper concerns the selection of CNN models for citrus disease detection. It offers a low-cost, portable solution that can help small-scale farmers in low-income areas reduce their crop losses.



Copyright ©2026 by authors and Galileo Institute of Technology and Education of the Amazon (ITEGAM). This work is licensed under the Creative Commons Attribution International License (CC BY 4.0).

I. INTRODUCTION

The citrus family has been one of the most important contributors to the global agricultural economy. They are the main source of nutrition, money, and jobs that the world needs. Among these crops, lemons, oranges, and limes have the highest commercial value and are the major contributors to the food industry and human diets. However, the productivity and profitability of citrus farming are under constant threat from pests and diseases. One of the most fatal diseases that leads to HLB (Huanglongbing) or citrus greening is the case. It is caused by a bacterium (*Candidatus Liberibacter asiaticus*) that is most commonly carried by the Asian citrus psyllid (*Diaphorina citri*). Eventually, the illness leads to the tree's deterioration, the yellowing of the leaves, the production of abnormal fruit, and, finally, the tree's death. Actually, in nations such as India, where citrus cultivation is a major activity, HLB is a major problem that threatens sustainable agriculture and adversely affects smallholder and commercial farms alike. The extent of HLB's spread has become so severe that it now threatens farmers' lives and leads to significant crop losses in both quantity and quality [1], [2].

The disease's qualities, i.e., its long environmental persistence and spread via insect vectors, only compound the difficulties of its control, thereby making it a priority for research and management across all citrus-growing areas worldwide. Traditional methods for identifying HLB rely mainly on experts conducting visual inspections, laboratory testing, or using diagnostic kits. Nevertheless, these approaches are labour-intensive, slow, and unsuitable for large-scale operations. The rapid and widespread distribution of HLB and the challenge of early detection necessitate an efficient, automated, and scalable solution to identify infected plants and help stop disease outbreaks. Most conventional methods have a low success rate in early detection of the disease, and visual symptoms of HLB may be similar to other physiological disorders, which will lead to an incorrect diagnosis and prolonged waiting time for treatment [3], [4]. Timely diagnosis of an illness at the earliest stage is vital for treatment, especially when symptoms are barely visible or even unknown. Deep learning has been identified as a novel, effective method for automating disease detection in agriculture, leveraging the latest artificial intelligence (AI) and computer vision. The introduction of deep convolutional neural networks (CNNs) has revolutionised image classification; as a result, deploying these techniques to detect HLB in citrus plants is widely regarded as a perfectly feasible task. These networks have a gift for spotting the tiniest details and image features. This way, they can distinguish between healthy and infected plants just as well. CNNs are now the leading models for plant disease recognition because they extract features directly from the original image, thereby drastically reducing the need for manual labour [5]. Apart from disease management, AI in agriculture has also facilitated plant detection and advanced decision-making for precision farming. This paper sheds the spotlight on the use of CNNs for farm diagnostics. HLB detection is the primary focus of this article, alongside other common citrus diseases researchers have studied.

The paper compares different CNN models. To provide a complete picture of a model's performance under extreme data shortages and resource constraints, these models include traditional convolutional architectures (AlexNet, VGG19) and newer DAG-based architectures (ResNet18, Inception-V3). The paper used a dataset of 955 images of Citrus leaves that were captured under the most stringent conditions. Besides healthy leaves, it also contained the first 10 festivals. Using 10-fold cross-validation, the paper achieved the highest sensitivity, demonstrating the HLB model's reliability. In all experiments with HLB, VGG19 led the way: it consistently achieved perfect sensitivity, even outperforming more complex architectures like Inception-V3 and ResNet18, which are typically regarded as high-performing on large datasets. It shows that simplified models can remain highly effective when data is limited, prioritising efficiency and accuracy over computational complexity alone. Besides that, CNNs have been quite a success story in small-data circumstances; this paper really advances the discussion. The present research contradicts this statement by showing that VGG19 can outperform its rivals; thus, the more complex, the better, is not true. Also, the discussion includes maximising diagnostic accuracy while minimising computational overhead, which is mainly important in resource-limited settings. Also, the research proposes that small-scale farmers, who are most at risk of suffering from diseases like HLB, may be given economical, easy-to-use, and portable instruments for disease identification, based on this scientific paper. The authors strongly encourage converting these CNN models into Mobile apps or miniature imaging devices, so the outcomes will not only be top-tier AI technologies but also practical agricultural solutions.

II. RELATED WORK

As stated by the Food and Agriculture Organisation of the United Nations (FAO), citrus is ranked as the second most globally grown fruit. Nevertheless, certain diseases caused by fungi, bacteria, pests, and viruses have been identified as lowering citrus production and quality in some areas [6]. Among the issues that have weakened the health of citrus trees, Huanglongbing (HLB), also called citrus greening, is the most virulent. Several areas where the disease has been discovered are South America, North America, Asia, and Africa. In India, HLB is a severe threat to citrus fruit production. The infected trees exhibit HLB symptoms, including leaf blistering, shoot yellowing, reduced fruit size, growth retardation, corky veins, root death, and dieback. These symptoms not only degrade the trees' health but also cause farmers to lose significant money, which in turn affects the citrus industry [7]. Several researchers have been working on machine-learning-based automation for plant leaf disease over the last few years. A recent overview of current image processing developments highlighted the importance of this technology in the development of practical, efficient devices for automated insect and disease identification, especially when combining machine learning and image processing [8]. Additionally, a Sparse Representation Classification (SRC) system for the automated recognition of cucumber diseases was proposed [9], as SRC's robustness to limited training data can enable more accurate and reliable disease detection. And a new characterisation of the dynamic system modulated model, i.e., the Group Method of Data Handling (GMDH), was introduced, along with the use of logistic regression to fit the dataset's statistical features [10]. This model significantly improves performance through its dynamic adjustments. Machine learning has recently been making strides towards developing hybrid models that combine deep learning methods with older techniques.

One of the foremost applications of deep learning and machine learning in the agricultural industry is advanced crop disease detection and diagnosis using image processing. With the goal of automated plant disease detection, the authors of [11] strongly believe that traditional and deep feature extraction methods can be used together, as they are complementary. Although deep learning techniques extract features automatically and traditional methods are better at classification, the authors of [12] propose a new technique that integrates K-means clustering and SVM for effective segmentation and classification of diseased areas on the leaf. Besides, their use of modified SVM methods with post-processing operations has led to a major improvement in classification accuracy [13]. DenseNet architectures, coupled with SVMs, have proven quite effective [14] for disease identification by leveraging deep feature extraction and ensemble learning. A complex hybrid multiclass SVM model has been proposed [15], which employs spatial fuzzy C-means clustering to correct spatial deviations and enhance classification accuracy. One of the chief reasons disease classification models can't be used across different crops is that they aren't directly transferable. Recently, localisation-based algorithms [16] have been proposed as a solution to this issue. These algorithms allow for more accurate and adaptable systems. Also, a leaf recognition technique based on Pulse Coupled Neural Networks (PCNN) was proposed [17], demonstrating that capturing detailed structural and textural features of leaves with PCNN yields significant gains in classification accuracy. Several papers [18] have highlighted the use of convolutional neural networks (CNNs) to, for example, detect diseases on citrus leaves in real-time; and one of the factors contributing to the success of mobile-based diagnostic tools is their accessibility and ease of use for the agricultural sector, which is presently going through significant changes.

The use of AlexNet [19] for categorising plant ailments also demonstrates that CNNs are best suited to highly organised agricultural datasets, thereby enabling high-accuracy, fast disease recognition. Image-based disease detection using deep learning techniques has been explored recently [20]. The application of transfer learning in agriculture has significantly enhanced plant disease classification, particularly when labelled datasets are scarce, leading to new ways of using pre-trained models. Another instance of feature-based classification methods evolving their capabilities is the use of colour histograms and textural descriptors to identify diseases in citrus plants. A comparative analysis of different machine learning methods applied to UAV-based multispectral images for citrus greening detection has revealed the importance of remote sensing technologies in agricultural monitoring. These exploratory studies emphasise the role of multispectral and hyperspectral imaging technologies in the early detection of plant diseases. In fact, a plant leaf disease recognition system based on deep learning techniques achieved a remarkable jump in classification accuracy [21]. Soon after, this research was further developed into smartphone-assisted systems [22] that allow for on-site diagnostics and support disease detection in farming practices. At the molecular level, real-time quantitative PCR (qPCR) is considered the gold standard for detecting *Candidatus Liberibacter* spp due to its unmatched sensitivity and specificity compared with other methods. qPCR detects the pathogen by targeting bacterial genes, such as 16S rRNA. The introduction of CRISPR-Cas12a-based diagnostic methods, such as DETECTR, has enabled the development of highly sensitive, rapid, and very convenient diagnostic kits for *Candidatus Liberibacter asiaticus* (CLAs) for field applications, thereby promoting early intervention [23]. Then again, digital droplet PCR (ddPCR) has proven to be an excellent tool for detecting ultra-low levels of infection, leading to improved diagnosis even in asymptomatic trees [24]. Techniques based on hyperspectral imaging (HSI) have been extremely effective for the non-invasive detection of HLB by highlighting physiological modifications in citrus trees that are invisible to the naked eye. Hyperspectral data, combined with machine learning algorithms, has played a significant role in enhancing diagnostic efficiency. At the very front line of plant disease diagnosis, CNNs process large datasets to extract highly complex features from images that even top human experts may not recognise [25].

A CNN method trained on images of citrus leaves achieved over 90% accuracy in identifying HLB symptoms [26], indicating the model's strength and potential in real-world settings. Transfer learning, which uses a pretrained model (e.g., VGGNet, ResNet, or Inception) as the starting point, has helped achieve results more quickly while reducing computational resources and training time. All these aspects still enable high accuracy even with a small amount of labelled data [27]. This is an excellent option for resource-constrained areas that lack large, well-annotated datasets. When combined with convolutional neural networks (CNNs), hyperspectral and multispectral imaging have paved the way for non-invasive HLB detection methods. Hyperspectral imaging first captures continuous spectral information, then processes it using a CNN that learns to distinguish healthy from diseased conditions. One combination of processing methods, accompanied by hyperspectral images, produced a CNN that accurately classified HLB-infected trees more than 97% of the time [28], demonstrating the potential of matching spectral data with deep learning for early disease detection without relying on visible symptoms. At the same time, steady enhancements in lightweight CNNs have enabled real-time HLB detection on mobile devices, striking the right balance between efficiency and accuracy without compromise, resulting in a model that is highly portable and mobile-friendly. This a lot eases on-site disease monitoring [29]. Although technology has made significant strides, challenges remain in implementing CNN-based disease detection systems. For instance, factors such as lighting, shadows, and background noise, which influence image quality, can vary from one image to another and may adversely affect the model's performance. To address these problems, the researchers have employed data augmentation and ensemble learning methods to enhance the generalisation capacity of the CNN models [30]. On top of that, the lack of HLB-oriented, standardised HLB-detection datasets is the main challenge in accumulating more extensive and diverse image repositories to support the spread of these technologies in this field.

III. MATERIALS AND METHODS

This section first describes in detail the use of pretrained models such as AlexNet and VGG19 for the primary purpose of classifying Huanglongbing (HLB) in citrus leaves. Additionally, a comparison of the performance of pretrained models such as ResNet-50 and Inception-V3 was conducted to determine the most suitable model for accurate HLB classification.

III.1 TYPES OF CITRUS HLB

Citrus Huanglongbing (HLB) can be classified into different categories depending on various factors. Heat-Tolerant Greening (HTG), caused by *Candidatus Liberibacter asiaticus* (CLAs), is a greening disease that prefers warm climates; thus, it warrants a thermal classification. On the other hand, Heat-Sensitive Greening (HSG), caused by *Candidatus Liberibacter africanus* (CLaf), is a greening disease that mostly occurs in cold regions. Zinc Deficiency-Induced Greening (ZD) is nutritionally similar to HLB but results from zinc deficiency, while Citrus Stubborn Disease (CSD), caused by *Spiroplasma citri*, mimics greening symptoms in Mediterranean climates. Citrus Blight (CB) is a disease that causes tree decline in the absence of psyllid vectors, and its cause remains unknown. Phytoplasma-Associated Greening (PAG) is a disease caused by wall-less bacteria transmitted by leafhoppers. In contrast, Seasonal or Cold-Induced Greening (CIG) is a disease caused by environmental stress, such as cold or drought. Cultural or Chemical Stress Greening (CSG) results from poor orchard management or chemical misuse. Mixed Infections Greening (MIG) involves co-infection by multiple pathogens, and Genotype-Specific Greening (GSG) reflects genetic susceptibility in certain citrus species or rootstocks, particularly sweet orange varieties.

III.2 DATA SETS

The primary dataset sources used in this paper are Kaggle and real field data, with a major emphasis on Huanglongbing (HLB) disease in citrus. It contains 955 images acquired under various natural conditions. These diverse images are used to classify the disease and assess the level of infection. Additionally, data augmentation was performed to both supplement the dataset and enhance the model's effectiveness, increasing the dataset to 5,200 images. The techniques of augmentation include mirror flipping, angular rotation, zooming, adjustments to brightness and contrast, conversion to grayscale, noise addition, and cropping.

The data consists of images of healthy citrus and citrus affected by HLB symptoms, enabling easy distinction between the two. During the severity assessment step, manual pixel-level annotations are created using the LabelMe tool [31]. The image pixels are categorized into four groups: healthy, HLB-infected (further divided into the main subclasses for each disease type), and background. Various colours are employed for better differentiation during the labelling. The dataset is split into 70% for training, 20% for validation, and 10% for testing the proposed technique.

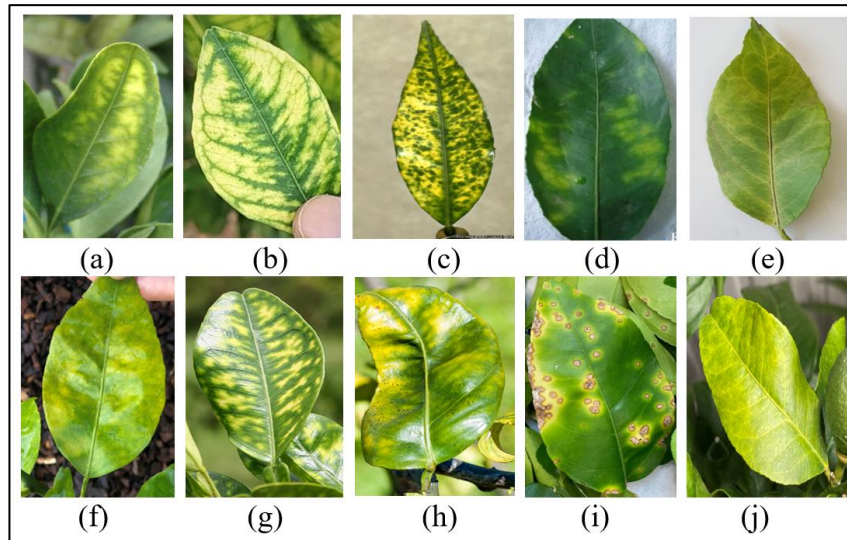


Figure 1: Sample original image dataset for infection with HLB.
(a) CSD (b) ZD (c) CB (d) PAG (e) CIG (f) CSG (g) MIG (h) GSG (i) HTG (j) HSG.
Source: Authors, (2026).

To comply with the input specifications of various models, the dataset is resized. For example, the dimensions of images are converted to (227x227x3) for AlexNet, (224x224x3) for ResNet and VGG19, and (299x299x3) for InceptionV3. After the images are resized, data augmentation techniques are applied. Although CNNs are highly effective, they can still suffer from overfitting, especially when trained on small datasets that limit their ability to generalise. One way to address this problem is to enlarge the dataset using label-preserving transformations. In other words, data augmentation refers to generating new data points by modifying the original images, thus increasing the number of training samples without collecting more.

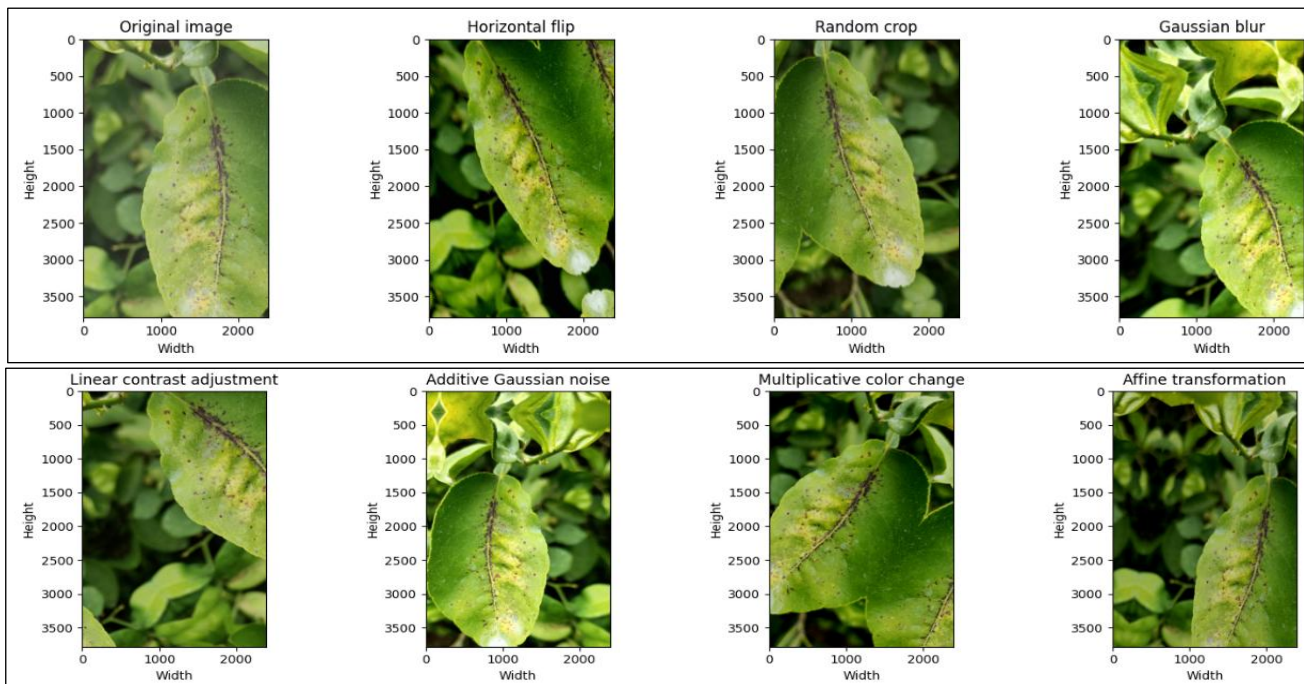


Figure 2: Sample Augmented Image dataset for Citrus HLB.
Source: Authors, (2026).

III.3 HARDWARE REQUIREMENTS AND SOFTWARE SPECIFICATIONS

The proposed structure is implemented using Python libraries and Keras coupled with TensorFlow. The experiments were conducted on a Windows 10 computer with an Intel i7 processor, 16 GB RAM, and a GPU with 8 GB of GDDR6 memory. Both training and testing of the models were performed using the Keras library, with TensorFlow 2.4.1 as the backend.

During hyperparameter tuning, 10 different configurations were tried, and each took nearly 7 hours to complete. In total, the time spent on training and testing the developed DCNN model was about 98 hours.

III.4 DEEP LEARNING MODELS & HYPERPARAMETERS

This research paper utilised deep learning models to optimise the performance of four different models. The study used four models: AlexNet, ResNet-50, VGG-19, and Inception-V3. This part of the paper introduces the background of the models employed in this research, their traits and functionalities.

III.4.1 AlexNet Architecture

The AlexNet model, introduced in 2012, revolutionised deep learning by achieving remarkable success in the ImageNet Large Scale Visual Recognition Challenge.

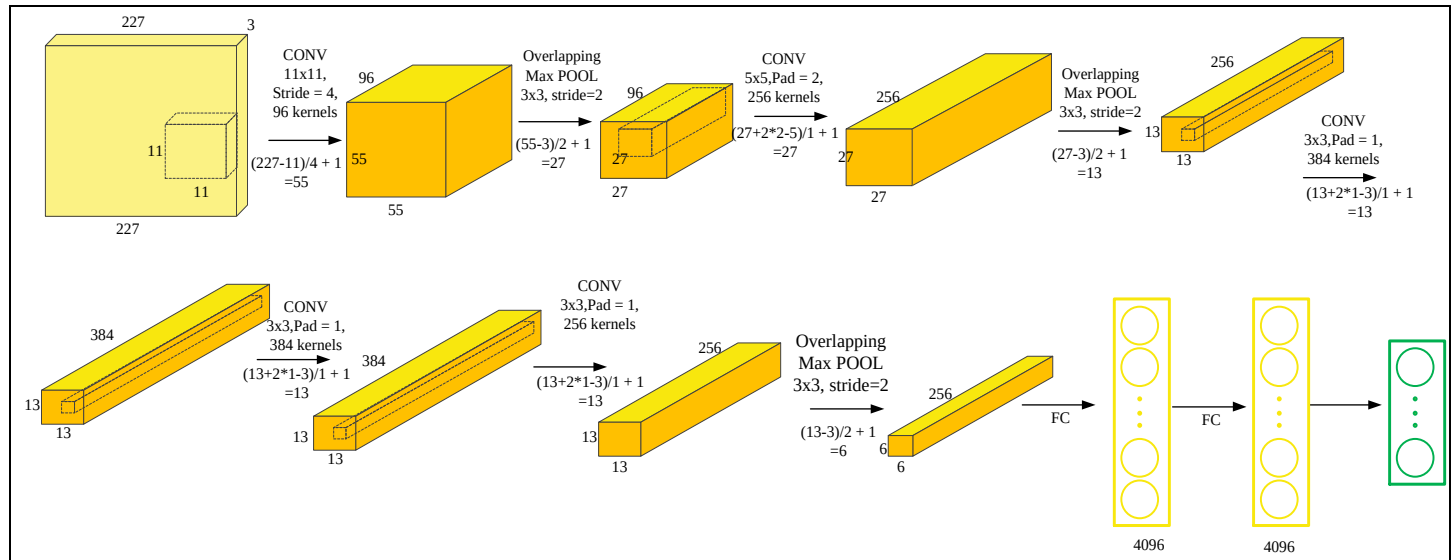


Figure 3: Architecture of AlexNet Classifier.

Source: Authors, (2026).

This CNN is designed to handle input images with a resolution of 227x227x3. The initial step of the network, feature extraction, is performed using a series of convolutional, pooling, and fully connected layers. The first convolutional layer applies 96 11x11 filters with a stride of 4, and the resulting features undergo overlapping max-pooling, which significantly reduces the spatial dimensions. Following the application of 256, 5x5 filters, the layer is subjected to max-pooling again. Further into the network, there are 5 more conv layers, and 3 of them are followed by max-pooling. Eventually, the output feature map has spatial dimensions 6x6 and a depth of 256 channels (6x6x256). The 2 fully connected layers, each with 4096 neurons and the ReLU activation function, receive the flattened output as input. Dropout regularisation is used in those layers to prevent overfitting. The last fully connected layer, which uses softmax activation, gives the probabilities over the 1000 categories. The most important AlexNet features that explain the rise in its performance are the use of the ReLU activation function, which makes it train faster; overlapping max-pooling, which makes it easier to generalise features; and GPU acceleration for training. The success of AlexNet was a major sign that deep learning was superior to traditional methods for image classification, and it paved the way for even more complex architectures. That is why the canonical AlexNet variant remains the foundation of transfer learning and feature extraction in computer vision.

III.4.2 ResNet-50 Architecture

The deep convolutional neural network model ResNet-50 is shown in Figure 4 and is widely used in image recognition and classification. It is part of the ResNet (Residual Network) family, which transformed the training of very deep networks by introducing residual learning. The idea of residual learning is that the network learns residual functions rather than direct ones, which helps address the vanishing gradient problem.

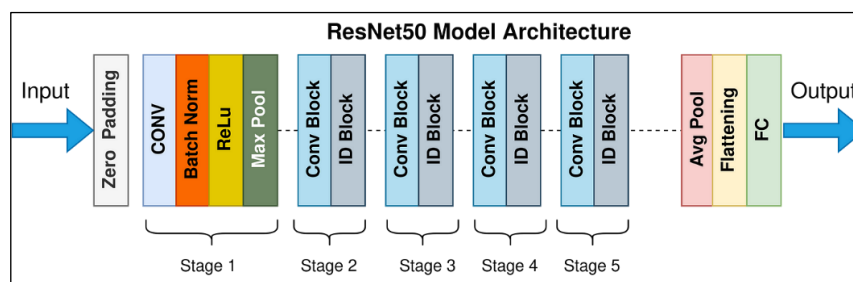


Figure 4: Architecture of ResNet50 Classifier.

Source: Authors, (2026).

The network takes its first input from an image data input layer. Here, zero-padding has been added to preserve the image's spatial dimensions. In Stage 1, the model consists of a convolutional layer that extracts low-level features from the image, followed by batch normalisation to stabilise and speed up training. A ReLU activation layer with non-linearity is added; max-pooling is also performed to downsample and, at the same time, keep essential features. Stages 2 to 5 are composed of a series of convolutional and identity blocks. A convolutional block is a residual block that uses skip connections, i.e. the output of the conv layers is added directly to the block's input. This addition helps the gradient flow smoothly through the network during backpropagation. This feature of deep ResNet alleviates degradation. Identity blocks preserve the input dimensions while learning features that are used more abstractly at a later stage. Throughout the network, the number of filters increases to handle increasingly complex patterns. The output layers finally consist of average pooling to reduce spatial dimensions, flattening to change feature maps into a 1D vector, and a fully connected layer for classification. The size of the fully connected layer depends on the number of classes. With its 50 layers, ResNet-50 finds a compromise between depth and computational efficiency. It is still widely used as the core structure for various image-based tasks, including image classification, object detection, and feature extraction for transfer learning. Due to its robust architectural design and innovative residual connections, it is widely regarded as a benchmark model in deep learning.

III.4.3 VGG19 Architecture

This diagram illustrates the structure of a convolutional neural network (CNN), specially designed for image processing and classification. It takes as input a single image of size 224×224 with 3 colour channels corresponding to RGB. The very first layers of the network consist of convolutional operations with the Rectified Linear Unit (ReLU) as the activation function. The main objective of these convolutional layers is to capture the image's spatial layout while shrinking its spatial dimensions and increasing the depth of the feature maps. For instance, the network passes on features of size $224 \times 224 \times 64$, $112 \times 112 \times 128$, $56 \times 56 \times 256$, and so on, along with the layer types. In this way, the net can depict both the minute and the large-scale features of the image. The feature maps at different depths of the hierarchy are next fed into a global average pooling layer at the $14 \times 14 \times 512$ stage. This layer considerably reduces the spatial dimensions by averaging feature values; hence, it provides dimensionality reduction while retaining the basic features for classification. Then the feature maps are passed to fully connected layers, which are the highest-connectivity point in the network, from which the extracted features are collected. These layers, together with the ReLU activation function, are responsible for generating complex decision boundaries by the model. Having a dropout layer with a rate of 0.5 in the network architecture can help reduce overfitting during training. It can randomly turn off half of the neurons. This regularisation method is the main source of the model's ability to generalise very well. The last layer of the model is a softmax layer that transforms the network's output into probabilities over each target class. In addition, the last layers are fine-tuned to adjust the pre-trained network for the specific classification task, which is why it achieves better performance.

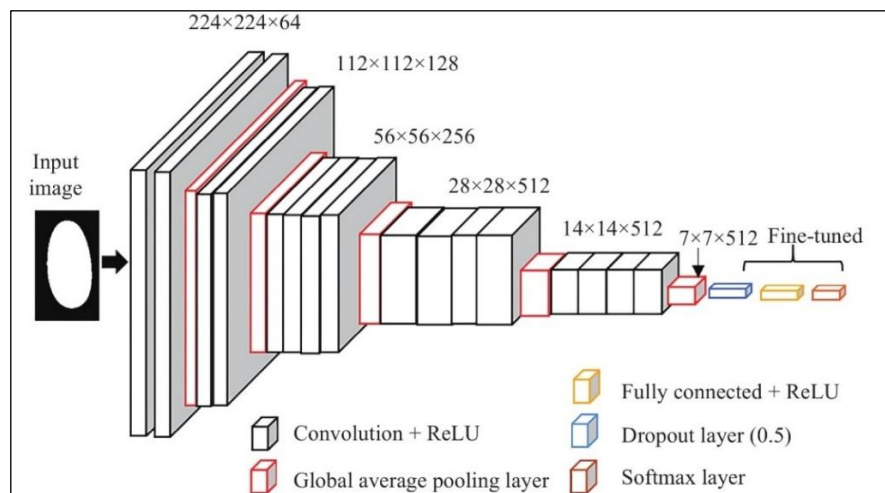


Figure 5: Architecture of VGG19 Classifier.

Source: Authors, (2026).

III.4.4 Inception V3 Architecture

The Inception-v3 model is among the top convolutional neural networks for image classification, and its effectiveness and efficiency have been widely acknowledged. The model receives an input image of size $299 \times 299 \times 3$ and produces an $8 \times 8 \times 2048$ feature map. It then classifies each input image into one of 1001 classes (or the background class). Belonging to the Inception family, it follows the design philosophy of making speed and accuracy trade-offs, like its ancestors, yet also introduces several changes that improve hardware efficiency and increase representational power. The crux of Inception-v3 lies in the Inception modules, which perform convolutions with different kernel sizes (e.g., 1×1 , 3×3 , and 5×5) concurrently. These modules merely don't output their outcomes to concatenate to gather very detailed features from the input data. The extensive use of 1×1 convolutions throughout the network helps drastically reduce computational cost while preserving the most important information. In addition to convolutions, the architecture uses several pooling layers, such as max and average pooling, which reduce the input's spatial dimensionality while accentuating salient features. The last phase of feature extraction produces a single, high-dimensional feature map ($8 \times 8 \times 2048$) that encodes a highly abstract, high-level representation of the input image. For classification purposes, the 3D feature map will undergo global average pooling, producing a 1D vector. Then, to get the class probabilities, a fully connected layer followed by a softmax activation will be used.

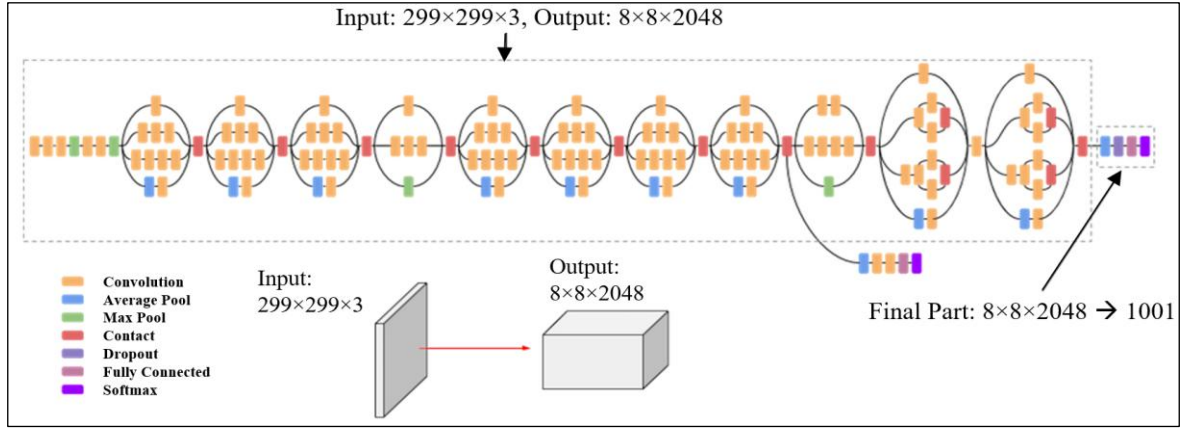


Figure 6: Architecture of InceptionV3 Classifier.
Source: [32].

Although Inception-v3 is quite old, it still introduces many new elements. One of those elements is factorised convolutions, which split large convolutions into smaller ones. For instance, a 5×5 kernel can be obtained by two consecutive 3×3 kernels, thereby reducing the number of calculations. Another revolutionary element is that auxiliary classifiers are incorporated into the training phase, thereby improving gradient flow and, as a result, eliminating the vanishing-gradient problem in deep layers. The model, with a proper combination of depth, accuracy, and computational efficiency, has become a standard solution for image classification, transfer learning, and feature extraction, among other tasks. So, hyperparameter tuning plays an important role in enhancing the performance of the proposed models. The results show that the model's accuracy measures quality. The summary of the hyperparameters used during training is provided below: AlexNet hyperparameters are in Table 1; Table 2 shows the parameters of ResNet and VGG19; and Table 3 presents the details for Inception-v3.

Table 1: Hyperparameters for AlexNet.

Parameter	Value
Image size	$227 \times 227 \times 3$
Batch size	64, 32, and 16
Number of epochs	100, 50, and 30
Optimizers	SGD, Adam, and RMSprop
Learning rate	0.01, 0.001, and 0.0001
Kernel size	11×11 , 5×5 , and 3×3
Number of hidden layers	5, 7, and 9
Loss function	Cross entropy

Source: Authors, (2026).

Table 2: Hyperparameters for ResNet50 and VGG19.

Parameter	Value
Image size	$224 \times 224 \times 3$
Batch size	128, 64, and 32
Number of epochs	150, 100, and 50
Optimizers	Adam, SGD, and RMSprop
Learning rate	0.001, 0.01, and 0.005
Kernel size	7×7 , 3×3 , and 1×1
Number of hidden layers	50, 19, and 34 (ResNet)
Loss function	Cross entropy

Source: Authors, (2026).

Table 3: Hyperparameters for InceptionV3.

Parameter	Value
Image size	$299 \times 299 \times 3$
Batch size	64, 32, and 16
Number of epochs	200, 150, and 100
Optimizers	Adam, RMSprop, and SGD
Learning rate	0.001, 0.0005, and 0.0001
Kernel size	3×3 , 5×5 , and 1×1
Number of hidden layers	48, 54, and 60
Loss function	Cross entropy

Source: Authors, (2026).

III.5 PROPOSED APPROACH FOR HLB DETECTION

III.5.1 Detection Scheme

Figure 7 displays the block diagram of the proposed HLB detection structure which contains three main parts: 1) image acquisition, capturing of leaf images that reflect real field conditions i.e. images taken from 13 acres citrus orchards of Gopanapally village, Podalakur Mandal, Nellore District, Andhra Pradesh, South India and also downloaded from Kaggle datasets; 2) segmentation (isolation of the leaf's inner parts); and 3) the CNN model i.e. classification which separates 10 types of HLB abnormalities apart from healthy image.

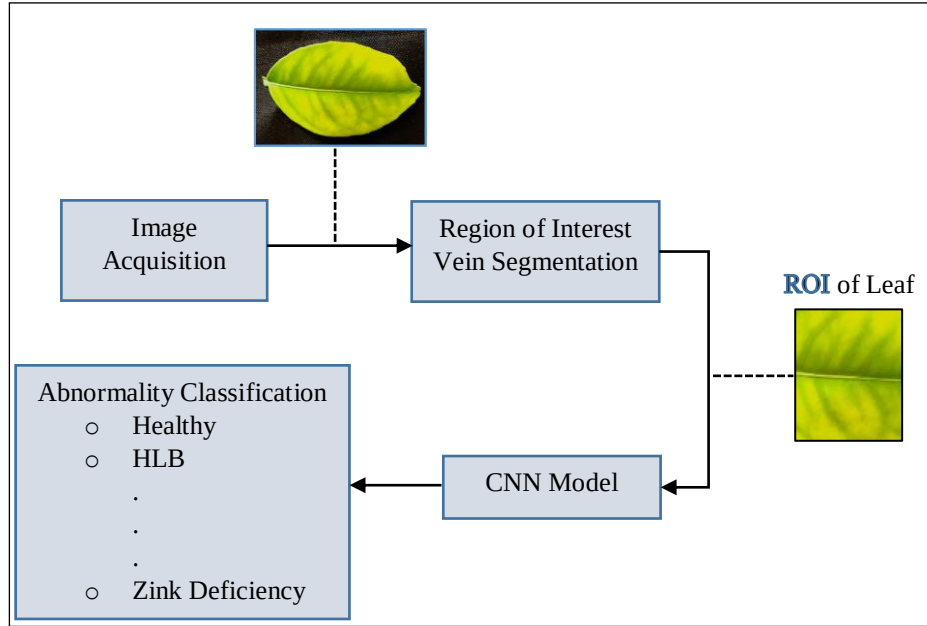


Figure 7: Image detection scheme for Citrus HLB classification.

Source: Authors, (2026).

III.5.2 Proposed Integrated Methodology

The strategy used for Huanglongbing (HLB) identification was a hybrid technique that merged leaf vein feature extraction with convolutional neural network (CNN) detection, as illustrated in Figure 8. The planned method combined domain-specific leaf features with the latest deep learning techniques to achieve precise disease classification. Below is a detailed explanation of each step of the algorithm:

Step 1. Leaf Image Acquisition

This begins by taking extremely clear photos of citrus leaves. These images work as the primary inputs of the subsequent process. Leaf sampling involves a fairly balanced selection of healthy and potentially diseased leaves, which are then used to create the training and testing datasets.

Step 2. Vein Segmentation

Leaf vein segmentation is performed on raw leaf images to isolate vein patterns. Segmentation is a way to keep only the major and minor veins of a leaf while discarding the lamina and other non-vein areas. This step is crucial as HLB-related leaf alterations are mainly visible in the veins. Here, the most important step is isolating veins from leaves. For this purpose, a thresholding technique is employed as described by this formula:

$$\text{Thresholding: } I_{seg}(x, y) = \begin{cases} 1, & \text{if } I(x, y) > T \\ 0, & \text{Otherwise} \end{cases}$$

Where T is the Threshold Value

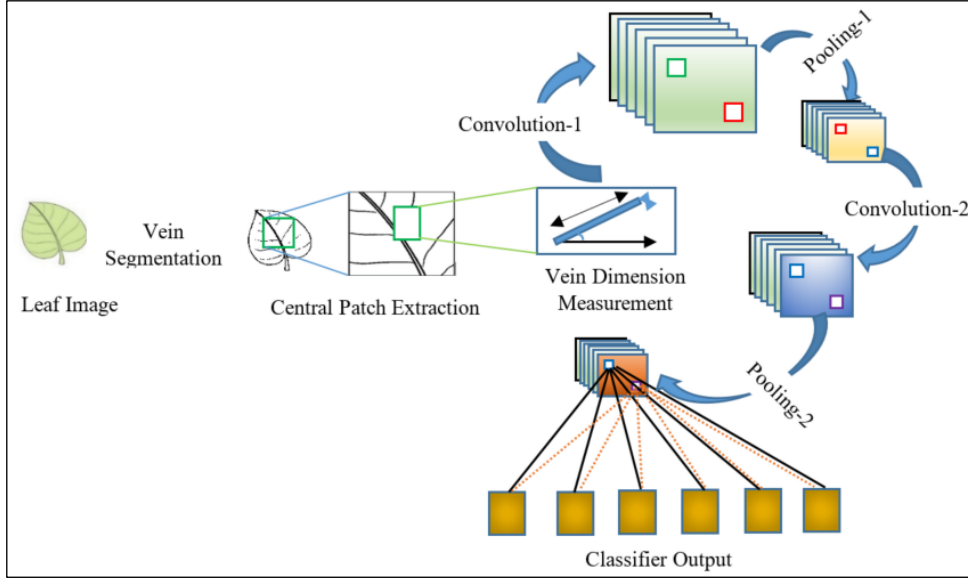


Figure 8: The proposed Integrated CNN Architecture model for HLB classification.
Source: Authors, (2026).

Step 3. Central Patch Extraction

Local vein structures have been segmented to obtain the central patch, which delimits a small area of high concentration. This particular patch of the veins contains fundamental features, such as where the veins branch and their width and density. The study has narrowed down the most vital area of the leaf vein structure by isolating it from the rest of the leaf surface, which is a source of noise. The central part of the vein structure is used to obtain a region of interest (ROI). The mathematical representation of ROI is like this:

$$P(x, y) = I_{seg}(x_i: x_f, y_i: y_f)$$

Where, (x_i, y_i) and (x_f, y_f) define the bounding box

Step 4. Vein Dimension Measurement

The extracted patch is further analysed to determine the vein dimensions, which are handcrafted features indicating the presence of HLB infection. These parameters include:

- Vein thickness.
- The directions of veins and the angles of the branches.
- Vein concentration and spacing.

$$\text{Length: } L = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Width: The distance is measured perpendicularly between two parallel vein edges.

$$\text{Angle: } \theta = \arctan\left(\frac{y_2 - y_1}{x_2 - x_1}\right)$$

These features are inspired by biology and provide important clues about the structural changes that occur as a result of HLB.

Step 5. Feature Extraction via CNN

The central patch is input to a convolutional neural network (CNN) for automatic feature extraction. The steps are:

Convolution Layer 1: Identifies simple visual elements such as lines, edges, and textures.

Pooling Layer 1: Reduces the sizes of the feature maps to save computational resources.

Convolution Layer 2: Dissects complex vein interactions and structural details at a higher level.

Pooling Layer 2: Further compacts the feature rewriting for the next steps.

$$O_{ij}^{(1)} = \sum_m \sum_n I_{(i+m)(j+n)} \cdot K_{mn}$$

Where K_{mn} is the kernel/filter

Pooling-1: Spatial dimensions are reduced; max-pooling is applied in this paper, which is represented by the formula

Convolution-1: Applies filters to detect basic patterns

$$P_{ij} = \max\{O_{ab} | a \in [i, i + s], b \in [j, j + s]\}$$

Convolution-2 and Pooling-2: Extract deeper features and further reduce the dimensions

Step 6. Classification

The features extracted are what a classifier is trained on; a fully connected neural network with a softmax output layer is the most common option. The classifier's results are a probability or a binary value indicating whether a leaf is healthy or HLB-infected. When it comes to multiclass problems, the classifier uses the softmax activation function defined by this formula,

$$P(C_k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Where z_k is the input to the k^{th} neuron in the final layer.

This research considered 10 classes (855 images) of HLB-infected image samples and 1 class of Healthy (100 images) for disease classification; the details are tabulated in Table 4.

Table 4: Citrus HLB characteristics and dataset details.

S No	Class	# Samples	Disease description
1	HTG	85	Heat-Tolerant Greening
2	HSG	100	Heat-Sensitive Greening
3	ZD	120	Zinc Deficiency-Induced Greening
4	CSD	80	Citrus Stubborn Disease
5	CB	80	Citrus Blight
6	PAG	80	Phytoplasma-Associated Greening
7	CIG	75	Cold-Induced Greening
8	CSG	75	Chemical Stress Greening
9	MIG	80	Mixed Infections Greening
10	GSG	80	Genotype-Specific Greening
11	HLT	100	Healthy

Source: Authors, (2026).

III.6 PERFORMANCE METRICS

Performance parameters evaluate the model's ability to learn and generalise on image data. These include classification metrics, loss functions, and other performance-related indicators specific to deep learning models.

III.6.1 Classification Metrics

These metrics measure the model's ability to classify data correctly. They are computed from the CNN's predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN}$$

$$\text{F1 Score} = 2 * \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Specificity (True Negativity Rate)} = \frac{TN}{TN + FP}$$

Where, TP = True Positive, TN = True Negative, FP = False Positive and FN = False Negative

III.6.2 Loss Function

The loss function indicates how well the model performs during training and validation. This research utilised the cross-entropy loss function, detailed in the equation below.

$$Loss = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \cdot \log(\hat{y}_{ij})$$

Where, y_{ij} is the ground truth and $\hat{y}_{ij}$ is the predicted probability for class j.

III.6.3 Model Specific Metrics

These are specific to the CNN model and the nature of the Task:

- Confusion matrix: A Visual representation of predicted values and actual labels, helping to assess the misclassification rates.
- IoU (Intersection over Union): Used in object detection and segmentation tasks to evaluate overlap between predicted and actual bounding boxes/Masks

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

- Dice Coefficient (F1 score for segmentation): Used for image segmentation

$$Dice = \frac{2 \cdot |A \cap B|}{|A| + |B|}$$

Where A is the predicted segmentation, and B is the ground truth.

IV. RESULTS AND DISCUSSIONS

This section presents the performance evaluation of the convolutional neural network (CNN) models, namely InceptionV3, ResNet50, AlexNet, and VGG19, for classifying Huanglongbing (HLB) and other citrus disorders using a dataset of 955 colour images of citrus leaves. The models were evaluated over multiple epochs, with the primary metrics being accuracy, loss, precision, recall, specificity, F1-score, Intersection over Union (IoU), and pixel accuracy.

IV.1 INCEPTIONV3

The two diagrams summarise the training and validation metrics for the InceptionV3 CNN model over 200 epochs. Figure 9 shows accuracy across epochs, whereas Figure 10 shows loss across epochs. Both aim to reveal the model's learning and ability to generalise to unseen data. In the accuracy diagram, training accuracy (blue line) gradually increases with training. Finally, this line reaches 1.0, indicating that the model has successfully learned the patterns in the training dataset. But validation accuracy (red line) fluctuates and stays at a lower level. Eventually, this line will fluctuate between 0.75 and 0.85. The gap between training and validation accuracies indicates a generalisation problem, a symptom of overfitting. Such behaviour is not unusual for complex architectures like InceptionV3, which can capture highly detailed patterns. Getting rid of overfitting may require applying different regularisation techniques, such as dropout, data augmentation, or early stopping, to improve accuracy on the validation set.

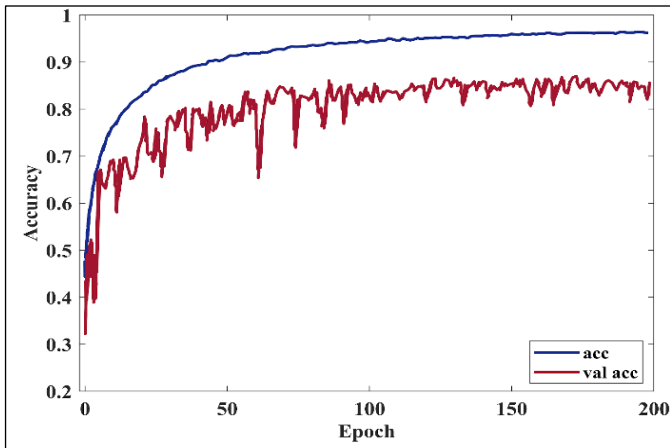


Figure 9: Training and validation accuracy curves with InceptionV3.
Source: Authors, (2026).

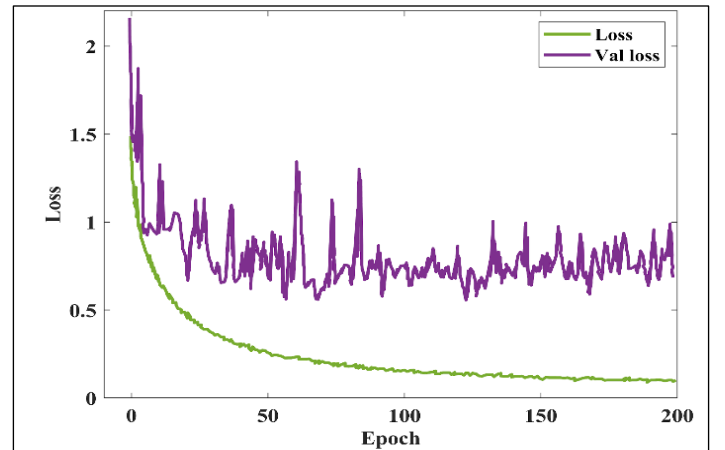


Figure 10: Training and validation loss curves with InceptionV3.
Source: Authors, (2026).

The loss graph conveys information beyond just the training loss (the green line), which continues to decrease in a steady, smooth manner, indicating good optimisation of the training objective. Whereas the validation loss (purple line) starts much higher and shows large fluctuations throughout the training process, one can see a very slight downward trend over time. The fact that the validation loss remains higher than the training loss indicates that the model is unstable on new data.

This instability might be a result of overfitting or some discrepancies in the distributions of the training and validation datasets. Tuning hyperparameters, using learning rate schedules, or simply verifying that the validation dataset is representative could help address the issue. The main point is that the InceptionV3 model is well-suited to the training data, as it achieved high training accuracy and low loss. But the variations in validation accuracy and loss, together with the large divergence between the training and validation metrics, suggest that the model is not very good at generalising.

IV.2 RESNET50

Figures 11 and 12 illustrate the training and validation performance of a ResNet model over 100 epochs. The progress made and the ability to generalise are evident. The training accuracy is illustrated by the blue curve in Figure 11, which shows a continuous increase; near epoch 80, at around 0.9, it indicates that the model is very well able to identify the patterns in the training data. Simultaneously, the validation accuracy (red curve) initially improves but remains below the training accuracy, indicating the challenge of generalising to new data. At about epoch 85, validation accuracy drops sharply, likely a sign of overfitting or an issue with the validation process, but it then goes back up. Figure 12 shows the training loss (green curve) and the validation loss (purple curve). Constant reduction in training loss indicates that the model is increasingly confident in the training data. Although validation loss is a bit erratic, it eventually decreases. The correlation between the rise in validation loss around epoch 85 and the fall in validation accuracy in Figure 11 strongly suggests overfitting or data inconsistencies.

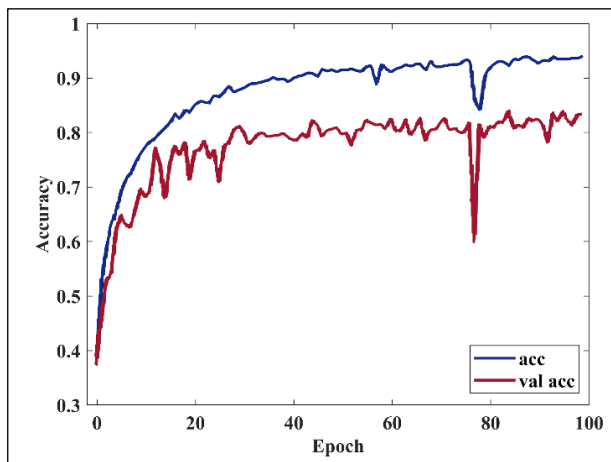


Figure 11: Training and validation accuracy curves with ResNet50.
Source: Authors, (2026).

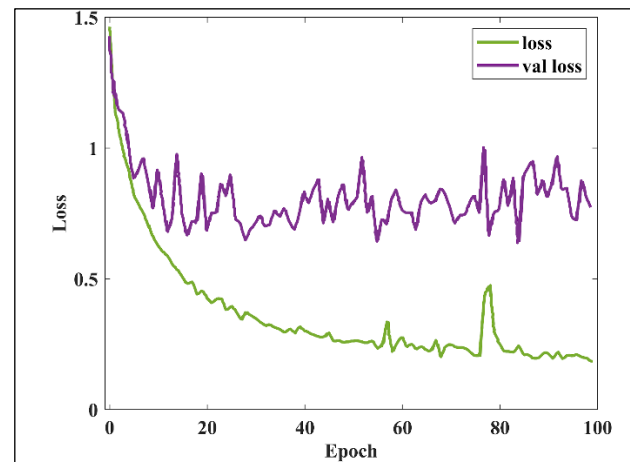


Figure 12: Training and validation loss curves with ResNet50.
Source: Authors, (2026).

IV.3 ALEXNET

Figures 13 and 14 show the AlexNet model's training and validation performance over 50 epochs. Whereas the former figure (Figure 13) presents accuracy plots, the latter (Figure 14) displays loss plots. These two figures together tell the story of the model's learning and its ability to generalise learned patterns to new, unseen data. The slowly, steadily increasing training accuracy (blue line) in Figure 13 eventually reaches 0.9. This shows that, apart from the model figuring out the relationships among the elements of the training data, it is also doing so very well. However, the validation accuracy (orange curve), lagging behind the training accuracy, levels off somewhere between 0.75 and 0.8 after 20 epochs. The gap between training and validation accuracies indicates a lack of model generalisation and is a hallmark of overfitting the training data. Since AlexNet is one of the very few models with such a strong capacity to learn detailed patterns, it also exhibits overfitting, as seen here. Regularisation methods such as dropout, weight decay, and data augmentation are well known to reduce overfitting and improve generalisation.

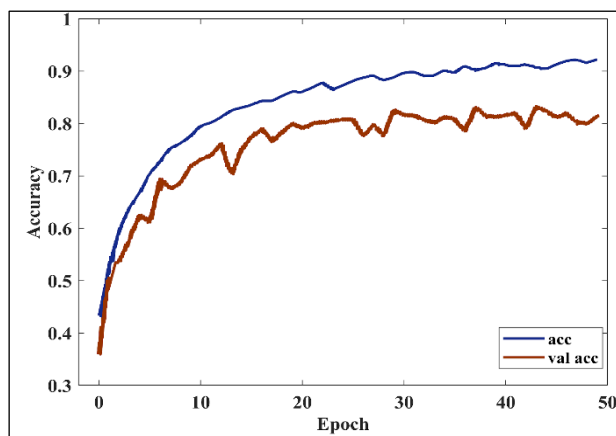


Figure 13: Training and validation accuracy curves with AlexNet.
Source: Authors, (2026).

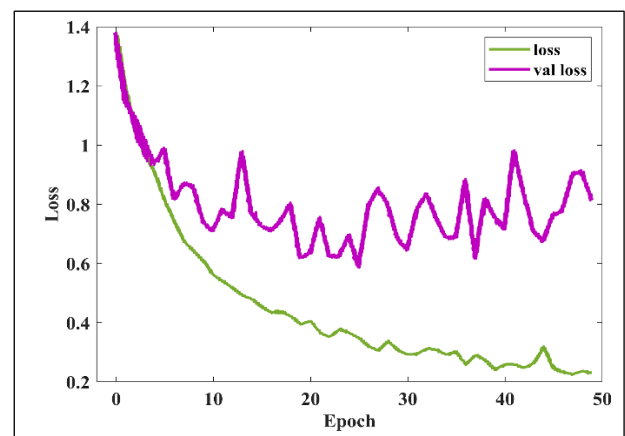


Figure 14: Training and validation loss curves with AlexNet.
Source: Authors, (2026).

The loss patterns are displayed in Figure 14. A decrease in the training loss (green curve) that occurs evenly and slowly over the epoch period indicates well-ordered training and results in a converged model. Conversely, the validation loss (purple curve) initially appears high and exhibits large fluctuations as it decreases, which is typical at later epochs when the model starts to overfit. Despite an overall decrease in the first epochs of training, the fluctuations remain quite strong even beyond epoch 10. These random variations in validation loss, plus the higher loss on the validation set than on the training set, reveal that the model is overfitting, a common problem the model neglects. The steady difference between training and validation loss is yet another sign that the model's generalisation performance needs improvement.

IV.4 VGG19

Graphs in Figures 15 and 16 show the training and validation metrics for two deep learning models, VGG19 and AlexNet, over 150 epochs. Such representations indicate the learning progression, the potential for generalisation, and the efficiency of model optimisation.

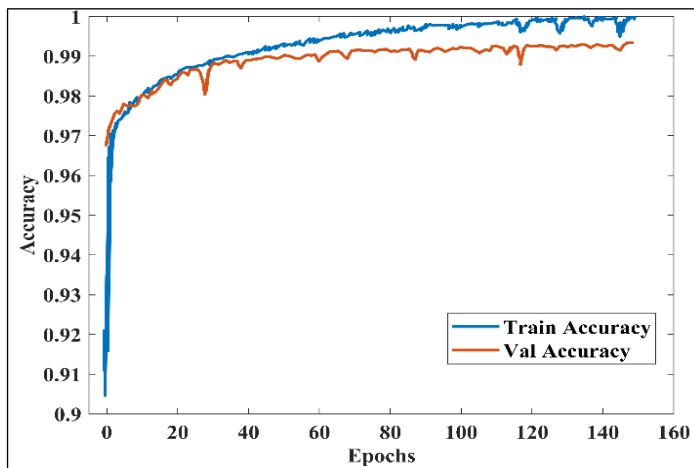


Figure 15: Training and validation accuracy curves with VGG19.
Source: Authors, (2026).

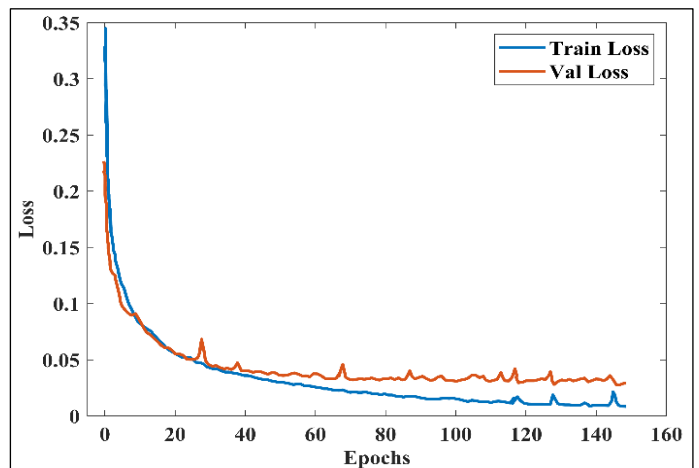


Figure 16: Training and validation loss curves with VGG19.
Source: Authors, (2026).

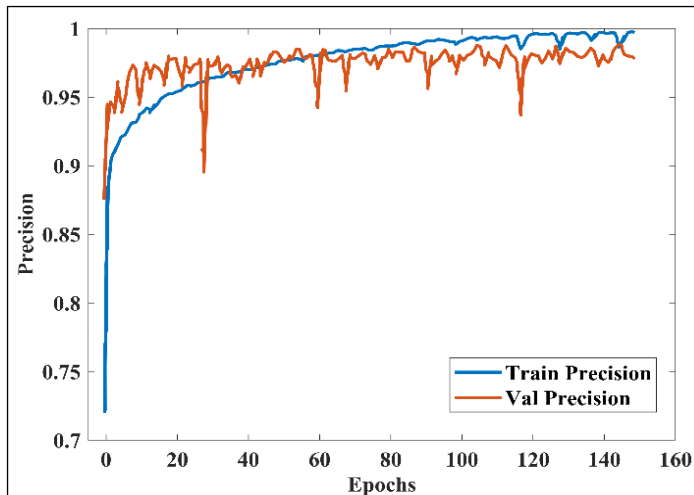


Figure 17: Training and validation precision curves with VGG19.
Source: Authors, (2026).

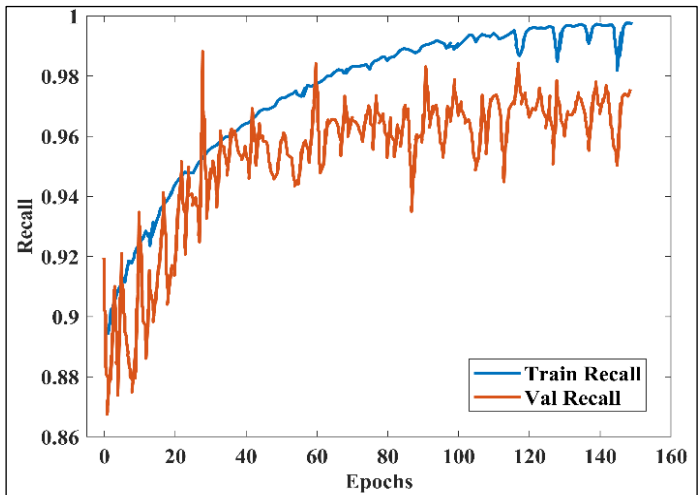


Figure 18: Training and validation recall curves with VGG19.
Source: Authors, (2026).

The VGG19 model achieves a final training accuracy of nearly 1.0 and a validation accuracy of approximately 0.98, with the difference between them very small. In the AlexNet example, the training loss decreases from 0.35 to 0.02, while the validation loss decreases from 0.35 to 0.03. Both outcomes indicate that the two models are good, with AlexNet being highly optimised and VGG19 achieving excellent accuracy. Still, the minor disparity between training and testing metrics suggests that the models' performance can be further improved through adjustments, such as tuning hyperparameters or incorporating additional regularisation techniques. Still, the small gaps between the training and validation scores suggest that the models' results could be improved by conducting experiments such as hyperparameter tuning or incorporating additional regularisation techniques. Figure 15 illustrates the performance of VGG19 using accuracy graphs. The analysis reveals that both the overall training and validation accuracies increased incrementally, reflecting the model's steady learning during training and validation. The training accuracy indicated by the blue line shows that the network reached 0.90 in the first epoch and, by the 150th epoch, had nearly achieved perfect training accuracy. This indicates that the model nearly flawlessly understands the patterns in the training set. Then, there is a certain pattern in validation accuracies: the orange line shows similar growth, though the training accuracies were a little higher. Starting from approximately 0.90, it finishes with nearly 0.98 at the very last epochs.

Given the minimal difference between training and validation accuracies, the VGG19 model can be considered to generalise well to unseen data. Although slight overfitting could be mitigated by methods such as data augmentation or stronger regularizers, it remains evident in the data. Figure 16 portrays the changes in losses for the AlexNet model, offering new perspectives on the optimisation steps. The blue curve representing the training loss gradually decreases from about 0.35 to nearly 0.02, reaching this level at 150 epochs. Based on the orange validation loss curve, the validation loss follows the same decreasing pattern observed in the training loss. In fact, it starts at around 0.35 and approaches 0.03 by the end of training. That the training and validation loss curves align closely indicates that the model generalizes well and, as a result, is not very susceptible to overfitting. Also, the loss in the initial epochs dropped steeply, further validating how quickly the model captured the essence of the dataset. Figures 17 and 18 present training and validation precision and recall charts for the VGG19 network over 150 training cycles, respectively. The steady rise in training precision shown in Figure 17 is encouraging, as it nearly reaches 1.0 at the end, the best possible score. Validation precision improves during the first few epochs. Still, from around epoch 30 onward, it behaves in a corkscrew manner through the remaining epochs, possibly due to data variability or minor overfitting. Similarly, in Figure 18, training recall increases continuously until it reaches a value close to 1.0 and remains at that level throughout the run, indicating that the model recognises only true positives. Validation recall is oscillating more widely, mainly because the number of false negatives varies with data complexity. These charts show the good and the bad of this model. While the model fit well in the training setting, the changes in validation results suggest there may be difficulties with the model's generalisation to unseen data.

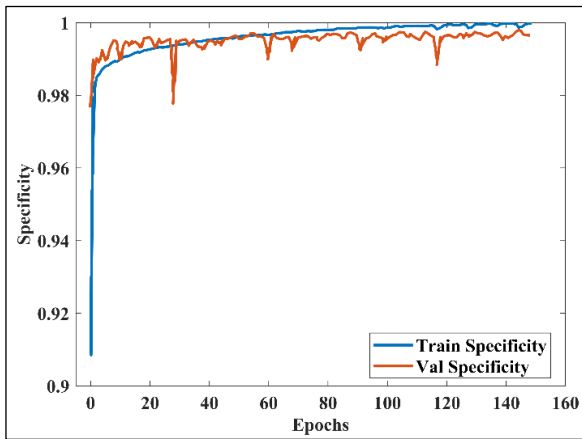


Figure 19: Training and validation specificity curves with VGG19.
Source: Authors, (2026).

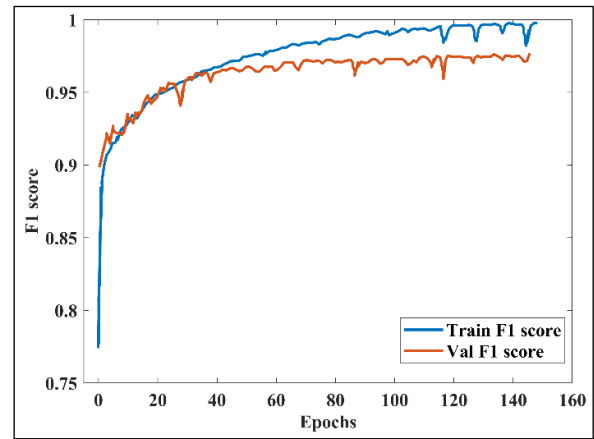


Figure 20: Training and validation F1 Score curves with VGG19.
Source: Authors, (2026).

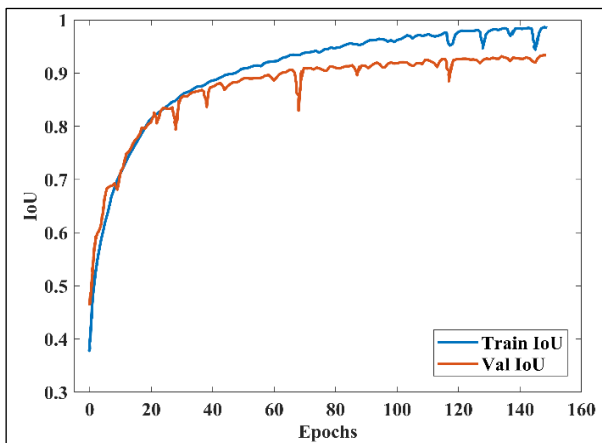


Figure 21: Training and validation IoU curves with VGG19.
Source: Authors, (2026).

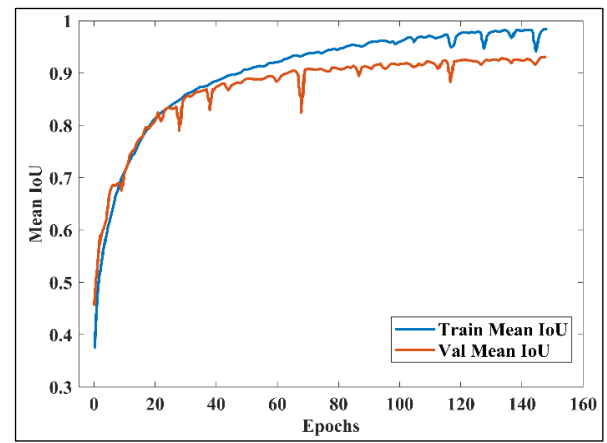


Figure 22: Training and validation mean IoU curves with VGG19.
Source: Authors, (2026).

Over 150 epochs, Figures 19 and 20 show the curves of specificity and F1-score for the VGG19 model on both the training and validation sets. In Figure 19, training specificity is indicated by the blue line. After an initial rapid increase, this line remains nearly constant at around 1.0, which corresponds to perfect prediction of the absence of positive cases. In contrast, the orange line representing validation specificity fluctuates slightly over time, likely due to differences between the validation and training data. In Figure 20, the training F1-score is consistently rising; towards the end, it is nearly 1.0, indicating that the model is well balanced between precision and recall. The validation F1-score is increasing but with ups and downs, possibly due to difficulty generalising the solution to unseen data. Figures 21 and 22 illustrate the training and validation IoU curves, as well as the mean IoU curve, for the VGG19 network trained for 150 epochs. Training IoU is steadily rising and ultimately settles near 1.0, indicating that the model can accurately segment the training data. However, the validation IoU, though it improves, exhibits fluctuations, which might imply the model is encountering different data or that learning has been less effective at generalising from seen to unseen examples. Also, the training mean IoU line practically touches 1.0 by the end. In contrast, the validation mean IoU shows a plateau, probably a sign of the model being overfit to the training data.

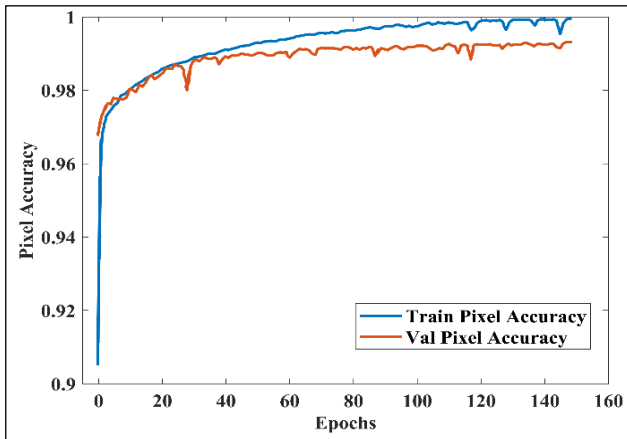


Figure 23: Training and validation Pixel Accuracy curves with VGG19.
Source: Authors, (2026).

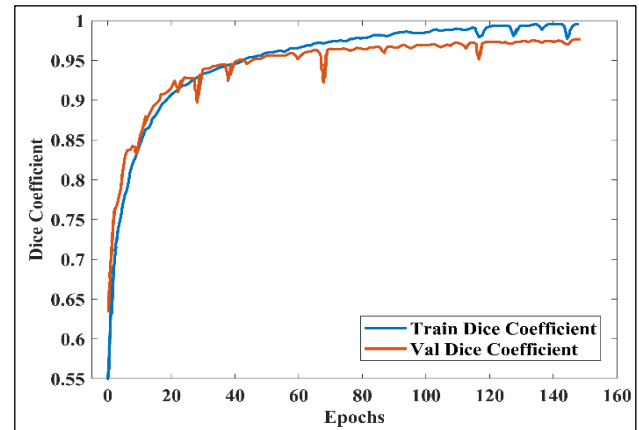


Figure 24: Training and validation Dice Coefficient curves with VGG19.
Source: Authors, (2026).

The VGG19 model's accuracy for pixel precision and Dice coefficients for segmentation are presented in Figures 23 and 24, respectively. Figure 23 shows that pixel accuracy continues to rise moderately, then remains elevated. Training accuracy close to 1.0 indicates that pixels have been classified very effectively; however, the slightly lower validation accuracy implies that the model is overfitting to some extent. Figure 24 shows the same pattern in the Dice coefficient: training quickly increases, reaches 1.0, plateaus, and at validation, the difference is minimal. Despite the small discrepancy, the model is well-generalized and achieves very high pixel accuracy and a high Dice coefficient, indicating excellent segmentation results.

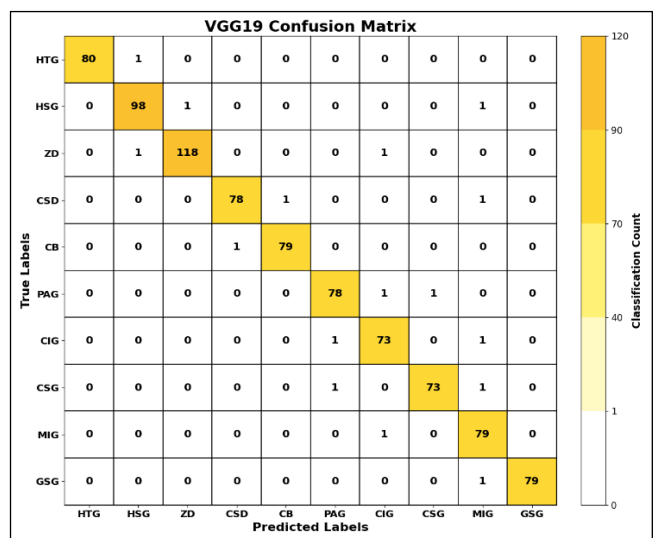
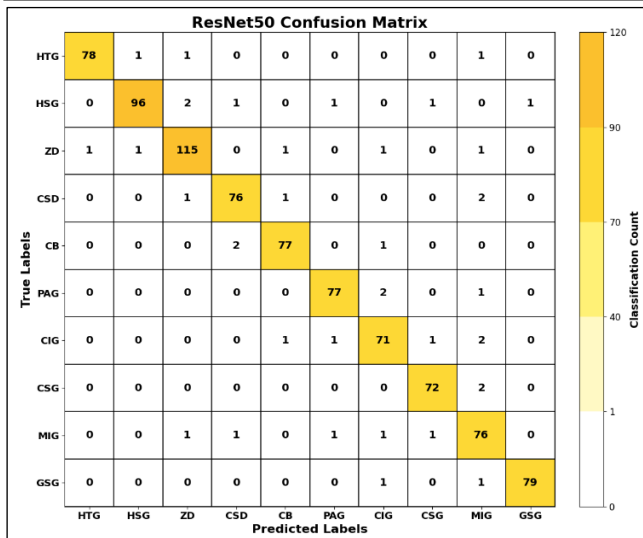
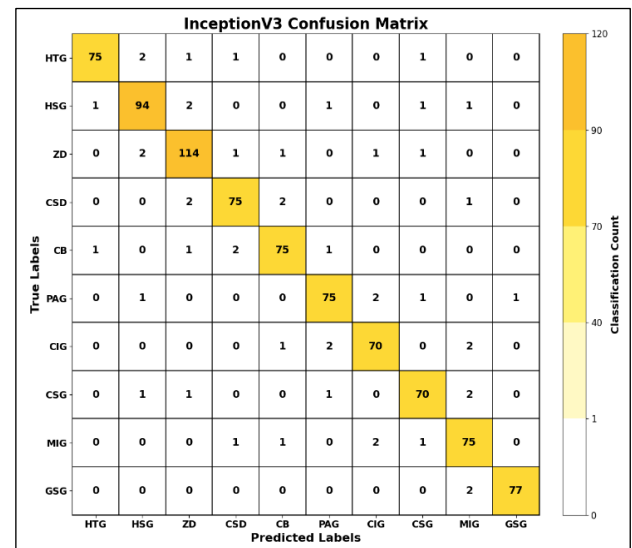
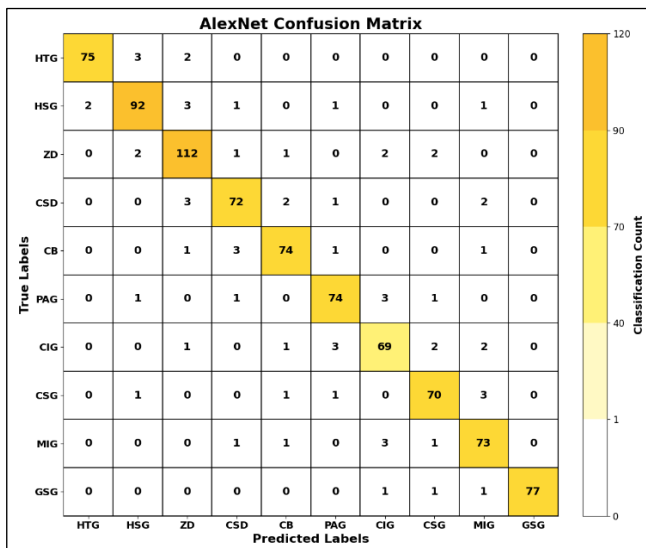


Figure 25: Confusion matrices for AlexNet, InceptionV3, ResNet50 and VGG19.
Source: Authors, (2026).

Figure 25 shows the confusion matrices of the four deep learning models for multi-class classification, i.e. AlexNet, InceptionV3, ResNet50, and VGG19. These matrices show in detail how each model performed on prediction across various classes. Diagonal cells correspond to the correct predictions, and the further away you are from the diagonal, the more errors that the models have made can be found. One can clearly see that ResNet50 and VGG19 are the two best performing. Table 5 presents a comprehensive comparison of computer vision methods for HLB detection in citrus leaf systems, including our proposed models. Our VGG19 models outperform other models in the literature for HLB disease classification.

Table 5: HLB detection methods reported in the literature with ‘n’ number of features and ‘c’ number of classes.

Imaging System	n	c	System Type	Features	Classifier	Accuracy	Ref.
Color Camera	600	6	CNN-based	Colour and texture	Softmax Layer	93%	[5]
Hyperspectral Camera	300	4	Conventional	Spectrum Features	SVM	89%	[8]
Fluorescence Spectroscopy	500	3	CNN-based	Learned Fusion Features	Auxiliary Classifier	96%	[9]
Polarization Camera	220	4	Conventional	Texture and Intensity	Random Forest	92%	[12]
Multispectral Camera	400	5	CNN-based	Learned Features	Softmax Layer	95%	[13]
Thermal Imaging Camera	350	4	Conventional	Intensity Statistics	SVM	90%	[14]
Colour and Hyperspectral Camera	800	3	Conventional	Texture and Spectrum Features	ANN	91%	[15]
Infrared Camera	450	2	CNN-based	Colour and Texture	Softmax Layer	94%	[16]
Fluorescence and Multispectral	550	6	CNN-based	Learned Features	Softmax Layer	97%	[18]
UV Imaging Camera	300	3	Conventional	UV Spectrum Analysis	SVM	88%	[19]
Laser-Induced Fluorescence	250	4	CNN-based	Fluorescence Patterns	ANN	94%	[20]
RGB-D Camera	400	5	Conventional	Depth and Colour Features	Random Forest	91%	[21]
Hyperspectral and Thermal	600	6	CNN-based	Multi-Modal Features	Softmax Layer	96%	[24]
NIR Imaging Camera	370	3	Conventional	Near-Infrared Features	Random Forest	90%	[26]
Fluorescence Hyperspectral	320	5	CNN-based	Learned Fusion Features	Auxiliary Classifier	95%	[28]
Terahertz Imaging Camera	450	2	Conventional	Terahertz Signal Analysis	SVM	87%	[29]
Multispectral Infrared	520	4	CNN-based	Multi-Modal Spectrum Features	Softmax Layer	92%	[30]
Enhanced RGB Camera	490	6	Conventional	Colour, Texture, and Shape	Bagged Tree	90%	[32]
Color Camera	955	10	CNN-based	Colour and Texture & Features	Softmax Layer with VGG19	98%	Our Models

Source: Authors, (2026).

V. CONCLUSIONS

A series of experiments on a dataset of citrus leaf images aimed at detecting Huanglongbing (HLB) disease and other citrus abnormalities was conducted using four CNN (convolutional neural network) models: InceptionV3, ResNet, AlexNet, and VGG19. In addition to tracking overall accuracy and loss as usual, the models' efficacy in the early detection of plant diseases was assessed using additional performance metrics, including precision, recall, specificity, F1-score, and Intersection over Union (IoU). The study indicated that all four model variants performed very well in terms of accuracy during the first training phase. The problem was that the models' generalisation capabilities were compromised, as shown by the gap between the training and testing metrics. The performance of VGG19 was the highest, achieving 98% accuracy; additionally, it had the smallest discrepancy between training and validation accuracy, further evidence that the model displays strong generalisation abilities.

InceptionV3, ResNet, and AlexNet have made significant advances in feature extraction from training data. However, their performance is still limited by overfitting, which limits their potential for generalisation to new data (i.e., they are less effective). So, to improve generalisation and reduce overfitting, one should strongly consider advanced regularisation techniques like dropout, data augmentation, and early stopping, as well as hyperparameter fine-tuning, among others. Still, given that the results of VGG19 serve as a premise, CNN models, particularly when paired with appropriate regularisation methods, are reliable detectors and will remain so, as they enable the automatic detection of citrus diseases. This will be a game-changer for small-scale, low-income farmers, for whom one of the main ways to avoid crop losses and increase their output is through timely disease detection.

VI. AUTHOR CONTRIBUTIONS

Conceptualisation: Nagineni Venkata Sireesha and Gillala Rekha.

Methodology: Nagineni Venkata Sireesha and Gillala Rekha.

Investigation: Nagineni Venkata Sireesha.

Discussion of results: Nagineni Venkata Sireesha and Gillala Rekha.

Writing – Original Draft: Nagineni Venkata Sireesha.

Writing – Review and Editing: Gillala Rekha.

Resources: Gillala Rekha.

Supervision: Gillala Rekha.

Approval of the final text: Nagineni Venkata Sireesha and Gillala Rekha.

VII. REFERENCES

- [1] Achor, D. S., et al., “Huanglongbing and its impact on citrus production”, *Frontiers in Plant Science*, 10, 1183, 2019, <https://doi.org/10.3389/fpls.2019.01183>.
- [2] Bani-Hani, S., et al., “Deep learning-based image processing for crop disease detection: A review”, *Computers and Electronics in Agriculture*, 179, 105840, 2020, <https://doi.org/10.1016/j.compag.2020.105840>.
- [3] Ferentinos, K. P., “Deep learning models for plant disease detection and diagnosis”, *Computers and Electronics in Agriculture*, 145, 311-318, 2018, <https://doi.org/10.1016/j.compag.2018.01.001>.
- [4] González, D., et al., “Development of a real-time detection system for Huanglongbing using deep learning techniques”, *Computers and Electronics in Agriculture*, 182, 105972, 2021, <https://doi.org/10.1016/j.compag.2021.105972>.
- [5] Mohanty, S. P., et al., “Using deep learning for plant disease detection and classification”, *IEEE Transactions on Applied Artificial Intelligence*, 12(1), 29-35, 2020, <https://doi.org/10.1109/TAI.2020.2970600>.
- [6] Ravindran, R., et al., “Deep learning-based identification of plant diseases using leaf images: A survey”, *Computers and Electronics in Agriculture*, 181, 105900, 2021, <https://doi.org/10.1016/j.compag.2021.105900>.
- [7] Saha, P., et al., “Deep learning for plant disease detection and classification: A comprehensive review”, *Journal of Computational Science*, 43, 101116, 2020, <https://doi.org/10.1016/j.jocs.2020.101116>.
- [8] Sharma, H. S., et al., “Smart agriculture systems and technology: AI applications for plant health monitoring”, *Agricultural Systems*, 170, 89-100, 2019, <https://doi.org/10.1016/j.agsy.2018.12.007>.
- [9] Wang, Z., Sun, X., Zhang, Y. et al, “Leaf recognition based on PCNN”, *Neural Computing & Applications*, 27, 899–908, 2016, <https://doi.org/10.1007/s00521-015-1904-1>.
- [10] Lawrence C. Ngugi, Moataz Abelwahab, Mohammed Abo-Zahhad, “Recent advances in image processing techniques for automated leaf pest and disease recognition – A review”, *Information Processing in Agriculture*, Volume 8, Issue 1, 2021, Pages 27-51, ISSN 2214-3173, <https://doi.org/10.1016/j.inpa.2020.04.004>.
- [11] Shanwen Zhang, Xiaowei Wu, Zhuhong You, Liqing Zhang, “Leaf image based cucumber disease recognition using sparse representation classification”, *Computers and Electronics in Agriculture*, Volume 134, 2017, Pages 135-141, ISSN 0168-1699, <https://doi.org/10.1016/j.compag.2017.01.014>.
- [12] Junde Chen, Huayi Yin, Defu Zhang, “A self-adaptive classification method for plant disease detection using GMDH-Logistic model”, *Sustainable Computing: Informatics and Systems*, Volume 28, 2020, 100415, ISSN 2210-5379, <https://doi.org/10.1016/j.suscom.2020.100415>.
- [13] K. Thaiyalnayaki, Christeena Joseph, “Classification of plant disease using SVM and deep learning”, *Materials Today: Proceedings*, Volume 47, Part 1, 2021, Pages 468-470, ISSN 2214-7853, <https://doi.org/10.1016/j.matpr.2021.05.029>.
- [14] F. A. P. Rani, S. N. Kumar, A. L. Fred, C. Dyson, V. Suresh and P. S. Jeba, "K-means Clustering and SVM for Plant Leaf Disease Detection and Classification," 2019 International Conference on Recent Advances in Energy-efficient Computing and Communication (ICRAECC), Nagercoil, India, 2019, pp. 1-4, doi:10.1109/ICRAECC43874.2019.8995157.
- [15] Thyagaraj., T. Y. Satheesha and S. Bhairannawar, "Plant Leaf Disease Classification Using Modified SVM with Post Processing Techniques," 2023 International Conference on Applied Intelligence and Sustainable Computing (ICAISC), Dharwad, India, 2023, pp. 1-4, doi:10.1109/ICAISC58445.2023.10201001.
- [16] S. Sarkar, J. Adhikari Ray, C. Mukherjee, S. Ghosh, J. N and C. L. K R, "Plant Leaf Disease Classification Based on SVM Based Densenets," 2023 International Conference on Advances in Computation, Communication and Information Technology (ICAICIT), Faridabad, India, 2023, pp. 636-641, doi:10.1109/ICAICIT60255.2023.10465886.
- [17] Santosh Kumar Sahu, Manish Pandey, “An optimal hybrid multiclass SVM for plant leaf disease detection using spatial Fuzzy C-Means model”, *Expert Systems with Applications*, Volume 214, 2023, 118989, ISSN 0957-4174, <https://doi.org/10.1016/j.eswa.2022.118989>.

- [18] Azlah, M.A.F.; Chua, L.S.; Rahmad, F.R.; Abdullah, F.I.; Wan Alwi, S.R. Review on Techniques for Plant Leaf Classification and Recognition. *Computers*, 2019, 8, 77. <https://doi.org/10.3390/computers8040077>.
- [19] Zheng, Y.-Y.; Kong, J.-L.; Jin, X.-B.; Wang, X.-Y.; Su, T.-L.; Zuo, M, "CropDeep: The Crop Vision Dataset for Deep-Learning-Based Classification and Detection in Precision Agriculture", *Sensors*, 2019, 19, 1058. <https://doi.org/10.3390/s19051058>.
- [20] Edna Chebet Too, Li Yujian, Sam Njuki, Liu Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification", *Computers and Electronics in Agriculture*, Volume 161, 2019, Pages 272-279, ISSN 0168-1699, <https://doi.org/10.1016/j.compag.2018.03.032>.
- [21] Utpal Barman, Ridip Dev Choudhury, Diganto Sahu, Golap Gunjan Barman, "Comparison of convolution neural networks for smartphone image based real time classification of citrus leaf disease", *Computers and Electronics in Agriculture*, Volume 177, 2020, 105661, ISSN 0168-1699, <https://doi.org/10.1016/j.compag.2020.105661>.
- [22] Yashwant Kurmi, Suchi Gangwar, "A leaf image localisation-based algorithm for different crop disease classification", *Information Processing in Agriculture*, Volume 9, Issue 3, 2022, Pages 456-474, ISSN 2214-3173, <https://doi.org/10.1016/j.inpa.2021.03.001>.
- [23] Chen, H.-C.; Widodo, A.M.; Wisnujati, A.; Rahaman, M.; Lin, J.C.-W.; Chen, L.; Weng, C.-E, "AlexNet Convolutional Neural Network for Disease Detection and Classification of Tomato Leaf", *Electronics* 2022, 11, 951. <https://doi.org/10.3390/electronics11060951>.
- [24] Mohanty SP, Hughes DP and Salathé M, "Using Deep Learning for Image-Based Plant Disease Detection", *Frontiers Plant Science*, 7, 1419, 2016, doi:10.3389/fpls.2016.01419.
- [25] Shafik W, Tufail A, De Silva Liyanage C, Apong Raahm, "Using transfer learning-based plant disease classification and detection for sustainable agriculture", *BMC Plant Biol.*, 2024, 24(1), 136. doi:10.1186/s12870-024-04825-y.
- [26] H. Ali, M.I. Lali, M.Z. Nawaz, M. Sharif, B.A. Saleem, "Symptom based automated detection of citrus diseases using color histogram and textural descriptors", *Computers and Electronics in Agriculture*, Volume 138, 2017, Pages 92-104, ISSN 0168-1699, <https://doi.org/10.1016/j.compag.2017.04.008>.
- [27] Yubin Lan, Zixiao Huang, Xiaoling Deng, Zihao Zhu, Huasheng Huang, Zheng Zheng, Bizhen Lian, Guoliang Zeng, Zejing Tong, "Comparison of machine learning methods for citrus greening detection on UAV multispectral images, *Computers and Electronics in Agriculture*", Volume 171, 2020, 105234, ISSN 0168-1699, <https://doi.org/10.1016/j.compag.2020.105234>.
- [28] Wadhwa, S.; Rani, S.; Kavita; Verma, S.; Shafi, J.; Wozniak, M, "Energy Efficient Consensus Approach of Blockchain for IoT Networks with Edge Computing", *Sensors* 2022, 22, 3733. <https://doi.org/10.3390/s22103733>.
- [29] Andreas Kamilaris, Francesc X. Prenafeta-Boldú, "Deep learning in agriculture: A survey", *Computers and Electronics in Agriculture*, Volume 147, 2018, Pages 70-90, ISSN 0168-1699, <https://doi.org/10.1016/j.compag.2018.02.016>.
- [30] Sai Reddy, B., Neeraja, S, "Plant leaf disease classification and damage detection system using deep learning models, *Multimedia Tools Applications*, 81, 2022, 24021–24040, <https://doi.org/10.1007/s11042-022-12147-0>.
- [31] Utpal Barman, Ridip Dev Choudhury, "Smartphone assist deep neural network to detect the citrus diseases in Agri-informatics, *Global Transitions Proceedings*, Volume 3, Issue 2, 2022, Pages 392-398, ISSN 2666-285X, <https://doi.org/10.1016/j.gltp.2021.10.004>.
- [32] E. Aswini and C. Vijayakumaran, "Auto Detector for Huanglongbing Citrus Greening Disease using YOLOV7," 2023 World Conference on Communication & Computing (WCONF), RAIPUR, India, 2023, pp. 1-6, doi:10.1109/WCONF58270.2023.10235043.